Data Science Writeup

Assignment 1: Logistic Regression on Iris Dataset

Objective

To train a Logistic Regression model on the Iris dataset to classify whether a given flower belongs to the species *Iris-setosa* or not, and to evaluate the model performance using accuracy, precision, and recall metrics.

Introduction

Logistic Regression is a supervised machine learning algorithm used for binary classification tasks. Unlike linear regression, which predicts continuous values, logistic regression predicts the probability of a data point belonging to a particular class. It uses the **sigmoid function** to map predictions to a range between 0 and 1, making it ideal for classification problems.

In this assignment, the **Iris dataset**—a well-known dataset in data science—will be used. The dataset contains measurements of sepal length, sepal width, petal length, and petal width for three species of iris flowers: *Iris-setosa*, *Iris-versicolor*, and *Iris-virginica*.

Our focus is on **binary classification**:

- Class 1: *Iris-setosa*
- Class 0: Not *Iris-setosa* (either *Iris-versicolor* or *Iris-virginica*)

The model's performance will be evaluated using:

- **Accuracy**: Overall correctness of the model.
- **Precision**: How many flowers predicted as *Iris-setosa* are actually *Iris-setosa*.
- **Recall**: How many actual *Iris-setosa* flowers the model correctly identified.

Materials and Methods

Tools and Libraries Used: Python 3.x, Pandas (data handling), NumPy (numerical operations), scikit-learn (model training & evaluation), Jupyter Notebook / Google Colab (execution environment)

Dataset:, **Iris dataset** from scikit-learn

Methods:

1. Data loading and preprocessing
2. Binary target variable creation (*Iris-setosa* vs not)
3. Train-test split

4. Model training using Logistic Regression
5. Model evaluation (accuracy, precision, recall)

Procedure (Pseudocode)

```
Step 1: Import necessary libraries (pandas, numpy, sklearn)
Step 2: Load the Iris dataset from sklearn.datasets
Step 3: Convert dataset into a Pandas DataFrame
Step 4: Create a binary target column:
        If species == 'Iris-setosa' → 1
        Else → 0
Step 5: Separate features (X) and target (y)
Step 6: Split the dataset into training and testing sets (e.g., 80% train, 20% test)
Step 7: Initialize Logistic Regression model
Step 8: Train the model using the training data
Step 9: Predict the labels for the test data
Step 10: Calculate evaluation metrics:
         - Accuracy
         - Precision
         - Recall
Step 11: Display the results
```

Results

Sample Output Metrics (Values may vary due to random split):

- Accuracy: **1.00** (100%)
- Precision: **1.00** (100%)
- Recall: **1.00** (100%)

Assignment 2: Decision Tree Classifier on Titanic Dataset

Objective

To load and preprocess the Titanic dataset, train a Decision Tree Classifier to predict passenger survival, and evaluate the model using a confusion matrix.

Introduction

The **Titanic dataset** is one of the most popular datasets for practicing classification problems in machine learning. It contains details about passengers aboard the RMS Titanic, such as age, gender, passenger class, fare, and survival status.

A **Decision Tree** is a supervised learning algorithm that splits data into branches based on feature values, eventually reaching a decision (class label) at leaf nodes. It uses measures such as **Gini Impurity** or **Entropy** to determine the best splits.

In this assignment:

1. We will handle **missing values** to ensure data completeness.
2. Encode **categorical variables** into numerical format for model compatibility.
3. Train a **Decision Tree Classifier** to predict survival (1 = Survived, 0 = Not Survived).
4. Use a **confusion matrix** to evaluate model performance by comparing actual vs predicted values.

The confusion matrix provides insight into **True Positives, True Negatives, False Positives, and False Negatives**, helping us understand where the model is performing well and where it may be making errors.

Materials and Methods

Tools and Libraries Used: Python 3.x, Pandas (data manipulation), NumPy (numerical computations), scikit-learn (model training, encoding, metrics), Matplotlib & Seaborn (plotting confusion matrix)

Dataset: Titanic dataset (train.csv from Kaggle or seaborn's built-in dataset)

Methods:

1. Data loading
2. Data cleaning (handling missing values)
3. Encoding categorical variables

4. Model training using Decision Tree Classifier
5. Model evaluation using confusion matrix

Procedure (Pseudocode)

```
Step 1: Import required libraries (pandas, numpy, sklearn, seaborn, matplotlib)
Step 2: Load the Titanic dataset
Step 3: Inspect data for missing values
Step 4: Handle missing values:
        - Fill 'Age' with median
        - Fill 'Embarked' with most frequent value
Step 5: Encode categorical columns ('Sex', 'Embarked') using label encoding or one-hot encoding
Step 6: Select feature columns (X) and target column (y = 'Survived')
Step 7: Split the data into training and testing sets (e.g., 80% train, 20% test)
Step 8: Initialize the Decision Tree Classifier
Step 9: Train the model on the training data
Step 10: Predict survival on the test set
Step 11: Generate and plot the confusion matrix using seaborn's heatmap
Step 12: Interpret the confusion matrix results
```

Results

Sample Confusion Matrix Output (Values may vary depending on split):

	Predicted No (0)	**Predicted Yes (1)**
Actual No (0)	94	15
Actual Yes (1)	23	47

Interpretation:

- **True Negatives (94)**: Correctly predicted passengers who did not survive.
- **True Positives (47)**: Correctly predicted passengers who survived.
- **False Positives (15)**: Passengers predicted as survived but did not.
- **False Negatives (23)**: Passengers predicted as not survived but actually survived.

The confusion matrix reveals both strengths and weaknesses of the model — while it identifies many survivors correctly, it also misses some (false negatives).

Assignment 3: Sentiment Analysis on Movie Reviews using Naive Bayes Classifier

---

Objective

To perform sentiment analysis on a dataset of movie reviews using the Naive Bayes classifier, classifying each review as either **positive** or **negative** based on the text.

Introduction

Sentiment Analysis is the process of determining the emotional tone of textual data — in this case, movie reviews. The goal is to automatically classify text into predefined categories such as *positive* or *negative* sentiment.

For this assignment, we use the **Naive Bayes classifier**, a probabilistic machine learning model based on **Bayes' Theorem**. It assumes that all features are independent of each other given the class label (the "naive" assumption).

The main steps in sentiment analysis with Naive Bayes involve:

1. **Text preprocessing** (removing punctuation, converting to lowercase, removing stopwords).
2. **Feature extraction** (converting text into numerical form, often using Bag of Words or TF-IDF).
3. **Model training** on labeled data.
4. **Prediction** and evaluation of model accuracy.

Naive Bayes is especially effective for text classification tasks because:

- It is computationally efficient.
- It works well with high-dimensional data.
- It can handle noisy data.

Materials and Methods

Tools and Libraries Used: Python 3.x, Pandas (data handling), scikit-learn (Naive Bayes model, feature extraction, train/test split), NLTK / re (text cleaning)

Dataset: Movie reviews dataset (e.g., IMDB movie reviews, or any labeled positive/negative review dataset)

Methods:

1. Data loading

2. Text cleaning and preprocessing
3. Converting text into numerical feature vectors (TF-IDF / CountVectorizer)
4. Model training with Naive Bayes classifier
5. Model evaluation (accuracy score)

Procedure (Pseudocode)

Step 1: Import required libraries (pandas, sklearn, nltk, re)
Step 2: Load the dataset of movie reviews with sentiment labels
Step 3: Preprocess text data:
- Convert text to lowercase
- Remove punctuation and special characters
- Remove stopwords
Step 4: Split the dataset into training and testing sets (e.g., 80% train, 20% test)
Step 5: Convert text into numerical vectors using TF-IDF Vectorizer
Step 6: Initialize the Naive Bayes classifier (MultinomialNB)
Step 7: Train the classifier on the training set
Step 8: Predict sentiment for the test set
Step 9: Evaluate model accuracy and generate classification report
Step 10: Display results

Results

Sample Output (values may vary depending on dataset and split):

- Accuracy: **0.86** (86%)
- Precision (Positive): **0.88**
- Recall (Positive): **0.85**
- Precision (Negative): **0.84**
- Recall (Negative): **0.87**

Assignment 4: Salary Prediction using Simple Linear Regression

---

Objective

To build a **Simple Linear Regression** model to predict the salary of employees based on their years of experience, using a dataset of 100 employees.

Introduction

Simple Linear Regression is a supervised machine learning algorithm used to model the relationship between two variables:

- **Independent variable (X)**: Predictor (Years of Experience)
- **Dependent variable (Y)**: Target (Salary)

The model assumes a linear relationship between the two variables and fits a straight line to the data: $Y = mX + c$

Where:

- Y = Predicted salary
- X = Years of experience
- m = Slope of the line (change in salary per year of experience)
- c = Intercept (salary when years of experience is 0)

The algorithm works by minimizing the **Mean Squared Error (MSE)** between the predicted and actual salaries.

In this assignment, we use **100 employee records** to train the model and then use it to predict salaries based on experience. This approach is widely used in HR analytics and workforce planning.

Materials and Methods

Tools and Libraries Used: Python 3.x, Pandas (data handling), NumPy (numerical operations), scikit-learn (model training and evaluation), Matplotlib & Seaborn (visualization)

Dataset: Salary dataset with 2 columns: Years of Experience, Salary (100 rows)

Methods:

1. Load and inspect dataset
2. Split into features (X) and target (Y)

3. Train-test split
4. Fit Simple Linear Regression model
5. Evaluate model performance (R^2 score, Mean Squared Error)
6. Visualize regression line

Procedure (Pseudocode)

Step 1: Import required libraries (pandas, numpy, matplotlib, sklearn)
Step 2: Load the dataset (Years of Experience, Salary)
Step 3: Separate features (X) and target (y)
Step 4: Split data into training and testing sets (e.g., 80% train, 20% test)
Step 5: Initialize the LinearRegression model
Step 6: Train the model on the training data
Step 7: Predict salaries for the test set
Step 8: Calculate evaluation metrics (R^2 score, Mean Squared Error)
Step 9: Plot scatter plot of actual data points
Step 10: Draw regression line based on model predictions

Results

Sample Output Metrics (values may vary):

- R^2 Score: **0.95**
- Mean Squared Error (MSE): **2.1×10^5**

Observation:
The model shows a strong positive linear relationship between years of experience and salary. The high R^2 score indicates that the model explains most of the variability in salary. The regression line fits the data well, making it a reliable predictor for salary estimation based on experience.

Assignment 5: House Price Prediction using Simple Linear Regression

---

Objective

To implement a **Simple Linear Regression** model that predicts house prices based on square footage and evaluate it using **Mean Squared Error (MSE)** and **R^2 score**. Additionally, to plot the regression line for visualization.

Introduction

In real estate analytics, predicting house prices based on property features is a common problem. **Simple Linear Regression** is an effective method for modeling the relationship between a single independent variable (**square footage**) and a dependent variable (**house price**).

The regression model assumes a linear relationship of the form: Price = m × (Square Footage) + c

Where:

- m = slope of the regression line (price change per additional square foot)
- c = intercept (base price when square footage is zero)

This approach can be used by real estate agents, property developers, and homebuyers to make informed decisions. In this assignment, we use a dataset containing house sizes and their corresponding selling prices to train and evaluate the model.

Materials and Methods

Tools and Libraries Used: Python 3.x, Pandas (data handling), NumPy (numerical calculations), scikit-learn (model training & evaluation), Matplotlib & Seaborn (data visualization)

Dataset: Dataset with two columns: Square Footage, Price

Methods:

1. Load dataset
2. Prepare features and target
3. Train-test split
4. Train Simple Linear Regression model
5. Predict house prices for the test set
6. Calculate **MSE** and **R^2** score
7. Plot regression line over actual data points

Procedure (Pseudocode)

Step 1: Import libraries (pandas, numpy, matplotlib, sklearn)
Step 2: Load the dataset (Square Footage, Price)
Step 3: Separate features (X) and target (y)
Step 4: Split the dataset into training and testing sets (e.g., 80% train, 20% test)
Step 5: Initialize the LinearRegression model
Step 6: Train the model using the training data
Step 7: Predict prices for the test set
Step 8: Calculate evaluation metrics:
- Mean Squared Error (MSE)
- R^2 score
Step 9: Plot scatter plot of actual prices vs square footage
Step 10: Plot regression line representing model predictions

---

5. Results

Sample Output Metrics (values vary depending on dataset):

- Mean Squared Error (MSE): $\mathbf{3.5 \times 10^8}$
- R^2 Score: **0.92**

Observation:
The high R^2 score suggests that the model explains most of the variance in house prices using square footage. The plotted regression line fits the data points closely, confirming the strength of the linear relationship.

Assignment 6: K-means Clustering on Tissue Gene Expression Data

---

Objective

To perform **K-means clustering** with K=7K = 7 on the tissue gene expression dataset, compare the resulting clusters to the actual tissue types, and observe how the results vary when the algorithm is run multiple times.

Introduction

Clustering is an **unsupervised learning** technique used to group similar data points based on their features. Unlike classification, clustering does not use labeled data during training.

K-means clustering is a popular algorithm that partitions the dataset into KK clusters by:

1. Initializing KK centroids randomly.
2. Assigning each data point to the nearest centroid.
3. Updating centroids as the mean of assigned points.
4. Repeating until centroids stabilize.

In this assignment, we use the **tissue_gene_expression** dataset, which contains measurements of gene expression levels across different tissue types. The dataset has labeled tissue types, allowing us to compare **unsupervised cluster assignments** to actual categories.

We set K=7K = 7 because the dataset contains seven distinct tissue types. Since K-means initialization is random (unless fixed with a seed), running the algorithm multiple times may lead to different cluster assignments.

Materials and Methods

Tools and Libraries Used: Python 3.x, Pandas (data handling), NumPy (numerical operations), scikit-learn (KMeans algorithm)

Dataset: tissue_gene_expression dataset (features: gene expression levels, label: tissue type)

Methods:

1. Load the dataset
2. Separate features and actual tissue type labels
3. Apply K-means clustering with K=7K = 7
4. Compare cluster labels to actual tissue labels in a table format
5. Repeat clustering multiple times and observe changes in assignments

---

Procedure (Pseudocode)

Step 1: Import necessary libraries (pandas, numpy, sklearn)
Step 2: Load tissue_gene_expression dataset
Step 3: Separate feature matrix X and target vector y (tissue types)
Step 4: Initialize KMeans model with K = 7
Step 5: Fit the model on X and get predicted cluster labels
Step 6: Create a comparison table showing:
- Cluster number
- Count of samples for each actual tissue type in that cluster

Step 7: Repeat the K-means clustering multiple times with different random states
Step 8: Observe how cluster assignments change between runs

Results

Sample Cluster vs Actual Tissue Type Table (example values):

Cluster	Brain	Liver	Kidney	Lung	Colon	Muscle	Skin
0	18	0	0	0	0	0	0
1	0	12	0	0	0	0	0
2	0	0	15	0	0	0	0
3	0	0	0	16	0	0	0
4	0	0	0	0	14	0	0
5	0	0	0	0	0	13	0
6	0	0	0	0	0	0	11

Assignment 7: FP-Tree Construction on Grocery Dataset

Objective

To build a Frequent Pattern Tree (FP-Tree) from the grocery dataset with a minimum support of 0.001 and a minimum confidence of 0.8, and to demonstrate how the tree evolves after processing each transaction.

Introduction

Frequent Pattern Mining is a key task in data mining, used to discover associations, correlations, or frequent patterns among a set of items in transactional data.
The **FP-Growth algorithm** is an efficient method for frequent itemset mining without generating candidate itemsets explicitly, unlike the Apriori algorithm.
An **FP-Tree** (Frequent Pattern Tree) is a compact data structure that stores transactions in a prefix-tree format. This enables pattern generation by recursively mining conditional FP-Trees.

In this assignment, we use a **grocery dataset** and construct an FP-Tree while applying **minimum support** and **minimum confidence thresholds** to filter patterns. The construction will show how the FP-Tree grows transaction-by-transaction, providing a better understanding of how the algorithm organizes items for efficient pattern extraction.

Materials and Methods

Tools & Libraries: Python 3.x, pandas – for data handling, mlxtend or custom FP-Tree implementation – for FP-Growth logic, collections – for frequency counting

Dataset: Grocery transactions dataset

Parameters: Minimum support = 0.001, Minimum confidence = 0.8

Procedure

Step 1: Load the grocery dataset into a DataFrame.
Step 2: Calculate the frequency of each item and filter items below the minimum support threshold.
Step 3: Sort items in each transaction in descending order of frequency.
Step 4: Initialize an empty FP-Tree with a root node (null).
Step 5: For each transaction:

- Insert the sorted items into the FP-Tree.
- If the branch already exists, increment the count.
- Otherwise, create a new branch and link the node in the header table.
  Step 6: Repeat Step 5 for all transactions to build the complete FP-Tree.
  Step 7: Extract frequent patterns by recursively mining conditional FP-Trees.
  Step 8: Apply the minimum confidence threshold to generate association rules.

Results

The FP-Tree was successfully constructed and evolved as follows:

- **Transaction 1:** Root → [Items inserted, counts updated]
- **Transaction 2:** New items merged with existing branches where applicable
- **Transaction 3+:** Tree grows, merging common prefixes and expanding unique paths

The final FP-Tree structure and the generated association rules indicate strong frequent itemsets meeting the given support and confidence thresholds.

Assignment 8: Apriori Algorithm on Grocery Dataset

Objective

To apply the Apriori algorithm on the grocery dataset with a minimum support of 0.001 and minimum confidence of 0.8, generate association rules, and identify the top 5 rules sorted by confidence, highlighting the strong ones.

Introduction

Association Rule Mining is a fundamental technique in data mining used to discover interesting relationships between variables in large datasets. One of the most popular algorithms for mining frequent itemsets and deriving association rules is the **Apriori Algorithm**.

In this assignment, we use the **Grocery dataset**, which contains transactions listing the items purchased by customers. Using the Apriori algorithm, we aim to find patterns of co-purchases that can be used for recommendation systems, inventory management, and market basket analysis.

The algorithm works by:

1. Identifying **frequent itemsets** that satisfy a minimum support threshold.
2. Generating **association rules** from these itemsets that satisfy a minimum confidence threshold.
3. Sorting and highlighting strong rules that have the highest confidence values.

Here, we set the **minimum support to 0.001** to capture even low-frequency but potentially useful patterns, and **minimum confidence to 0.8** to ensure the rules are reliable.

Materials and Methods

Tools & Libraries: Python 3.x, pandas (for data manipulation), mlxtend (for Apriori and association rule generation), matplotlib/seaborn (for optional visualization)

Dataset: Grocery dataset containing transaction-level purchase records.

Procedure

Step 1: Import necessary libraries (pandas, mlxtend.frequent_patterns).
Step 2: Load the grocery dataset into a DataFrame.
Step 3: Convert transaction data into a one-hot encoded format suitable for the Apriori algorithm.

Step 4: Apply the Apriori algorithm with min_support = 0.001 to find frequent itemsets.
Step 5: Generate association rules from the frequent itemsets using min_confidence = 0.8.
Step 6: Sort the rules in descending order of confidence.
Step 7: Select the **top 5 association rules** and highlight the strong ones (confidence = 1.0 or very close to it).
Step 8: Display the rules with their support, confidence, and lift values.

Results

- Frequent itemsets with support ≥ 0.001 were identified.
- Association rules with confidence ≥ 0.8 were generated.
- The top 5 rules (sorted by confidence) were listed, with strong rules clearly highlighted.
- The rules indicate strong correlations between certain grocery items, useful for product placement and recommendations.

Assignment 9: Market Basket Analysis using FP-Growth Algorithm

---

Objective

To perform Market Basket Analysis using the FP-Growth algorithm on a retail dataset, extract frequent itemsets, and generate association rules to identify purchasing patterns that can help improve sales strategies.

Introduction

Market Basket Analysis is a popular data mining technique used in retail to discover relationships between items frequently purchased together. The FP-Growth algorithm is an efficient method for mining frequent patterns without candidate generation. Unlike Apriori, it compresses the dataset into an FP-tree structure and then extracts frequent itemsets directly.

By generating association rules from these frequent itemsets, we can identify patterns such as "customers who buy bread also tend to buy butter." These insights can be applied in cross-selling, store layout design, and targeted marketing campaigns.

Materials and Methods

Materials/Tools Used: Python 3.x. Libraries: pandas, mlxtend (for FP-Growth and association rules), Retail transaction dataset

Methodology:

1. Load and explore the retail dataset.
2. Convert transaction data into a one-hot encoded dataframe.
3. Apply the FP-Growth algorithm with a predefined minimum support threshold.
4. Generate frequent itemsets from the FP-tree.
5. Derive association rules using minimum confidence.
6. Analyze and interpret the top purchasing patterns.

Procedure (Pseudocode)

Step 1: Load retail transaction dataset into a pandas DataFrame.
Step 2: Preprocess data — remove duplicates and format into a list of transactions.
Step 3: Apply one-hot encoding to convert transaction data into binary format.
Step 4: Run FP-Growth with min_support (e.g., 0.005) to find frequent itemsets.
Step 5: Generate association rules using min_confidence (e.g., 0.6).

Step 6: Sort association rules by confidence in descending order.
Step 7: Highlight strong rules (high support & high confidence).
Step 8: Present top rules for decision-making.

Results

Observations:

- Frequent itemsets identified show strong co-occurrence patterns.
- Association rules reveal strong relationships between products often purchased together.
- Example rule (hypothetical): {Bread} → {Butter} with 0.62 confidence and 0.007 support.

Insights:

- These rules can be used to improve product placement in stores.
- Marketing campaigns can target customers with complementary product offers.

Assignment 10: Create a Stacked Bar Chart of Monthly Revenue by Region

---

Objective: To visualize revenue distribution across different regions for several months using a stacked bar chart, which allows comparison of total revenue and regional contributions over time.

Introduction: A stacked bar chart is a type of data visualization where each bar represents the total value of a category, and the segments (or "stacks") within the bar represent the breakdown of the category into subgroups.
In this task, we have monthly revenue data for three regions: **East**, **West**, and **North**. The chart will help to quickly identify patterns, trends, and relative performance of the regions over the months **March to July**.

Materials and Methods

Tools & Libraries: **Python**, matplotlib library for plotting, numpy (optional) for data handling

Data: **Months:** ["Mar", "Apr", "May", "Jun", "Jul"], **Regions:** ["East", "West", "North"], **Colors:** ["green", "orange", "brown"]

Procedure (Pseudocode)

Step 1: Import required libraries (matplotlib.pyplot).
Step 2: Define the months, regions, and revenue data for each region.
Step 3: Choose distinct colors for each region for better visibility.
Step 4: Create the first stacked bar segment for Region East.
Step 5: Create the second stacked bar segment (Region West) stacked on top of East.
Step 6: Create the third stacked bar segment (Region North) stacked on top of West.
Step 7: Add chart title, axis labels, and a legend to indicate regions.
Step 8: Display the stacked bar chart.

Results

- The stacked bar chart shows how total monthly revenue is distributed among the three regions.
- Color-coded segments (green, orange, brown) indicate the revenue contribution of East, West, and North respectively.
- The legend helps identify each region's corresponding color.

Assignment 11: Read the file moviesData.csv to Create a Bar Chart of critics score for the First 10 Movies

---

Objective: To read movie data from a CSV file, extract the critics_score of the first 10 movies, and visualize the results as a bar chart, saving the plot for future reference.

Introduction: A bar chart is a common visualization for comparing values across categories. In this assignment, the categories are **movie titles**, and the values are their **critics' scores** from the dataset moviesData.csv. This visualization will help identify which movies received higher or lower scores from critics in the selected range.

Materials and Methods

Tools & Libraries: **Python**, pandas for reading and handling CSV data, matplotlib for creating and saving the chart

Data: **Source file:** moviesData.csv, **Column of interest:** critics_score, **Number of movies:** First 10 records from the dataset

---

Procedure (Pseudocode)
Step 1: Import the required libraries (pandas and matplotlib.pyplot).
Step 2: Load the CSV file moviesData.csv into a DataFrame.
Step 3: Select the first 10 rows of the DataFrame.
Step 4: Extract the movie titles and their corresponding critics_score values.
Step 5: Create a bar chart where the x-axis represents movie titles and the y-axis represents critics' scores.
Step 6: Rotate x-axis labels for better readability.
Step 7: Add chart title, x-axis label, and y-axis label.
Step 8: Save the bar chart to a file (e.g., critics_score_chart.png).
Step 9: Display the plot (optional).

Results

- The bar chart clearly shows the critics' scores for the first 10 movies in the dataset.
- Titles are displayed along the x-axis, with scores on the y-axis.
- The plot is saved successfully for later use.

Assignment 12: Use the Dataset mtcars and Create a Boxplot for mpg and cyl Columns

---

Objective

To create a boxplot visualization using the mtcars dataset, showing the distribution of miles per gallon (mpg) values for different cylinder (cyl) categories.

Introduction

A **boxplot** is a statistical graphic that displays the distribution of numerical data through five-number summaries: minimum, first quartile, median, third quartile, and maximum. It is particularly useful for identifying **spread, central tendency, and outliers**.
In this task, the mtcars dataset will be used to compare mpg values across different cylinder counts (cyl).

Materials and Methods

Tools & Libraries: **Python**, Built-in mtcars dataset , Visualization library (matplotlib / seaborn in Python)

- **Dataset:** mtcars
- **Columns of Interest:** mpg → Miles per gallon (numeric), cyl → Number of cylinders (categorical)

Procedure (Pseudocode)

Step 1: Load the mtcars dataset into a DataFrame.
Step 2: Treat cyl as a categorical variable for grouping.
Step 3: Use a boxplot function to plot mpg (y-axis) against cyl (x-axis).
Step 4: Add appropriate axis labels ("Number of Cylinders", "Miles per Gallon") and a chart title.
Step 5: Style the plot for clarity (grid lines, colors, etc.).
Step 6: Display the plot.

Results

- The boxplot displays the median, quartiles, and potential outliers of mpg values for each cylinder category.
- A visible trend may be observed (e.g., higher cylinder count generally results in lower mpg).
